

Informatica 2.2 / Computational Intelligence

Genetic Programming

Marco S. Nobile, Ph.D.

Bachelor's Degree in Engineering Physics

Ca' Foscari University of Venice

A.Y. 2025-2026



Università
Ca' Foscari
Venezia

Outline

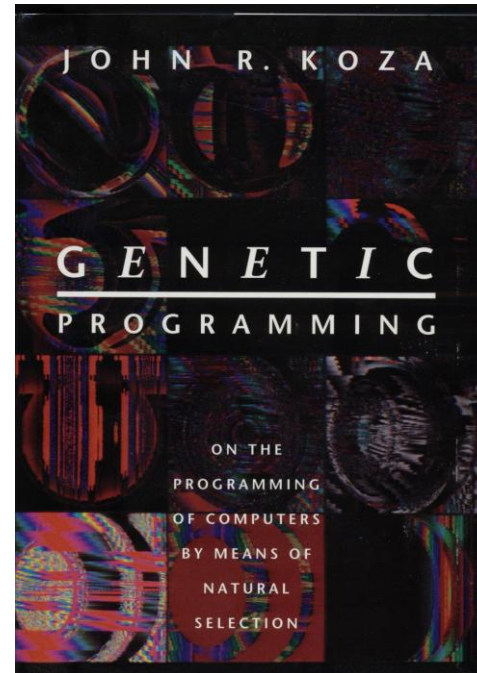
- Evolutionary Computation (reprise)
- Evolving programs: Genetic Programming
- Representing programs
- Genetic operators, closure, sufficiency
- Killer applications
 - Symbolic regression
 - Evolving Machine Learning pipelines (AutoML)

Evolutionary algorithms

- The idea of **exploiting evolutionary processes** to **find the optimal solution** to a complex computational problem (e.g., model calibration)
- One example of evolutionary approach is a **Genetic Algorithm (GA)**, created by **John Holland** in the 70s (although Turing himself suggested the idea in the 50s)
- GAs are **meta-heuristics**: they can be used to tackle **any optimization problem**
- A GA can solve **a given instance** of a problem (e.g., calibration of a model)

John Koza

- PhD student of John Holland

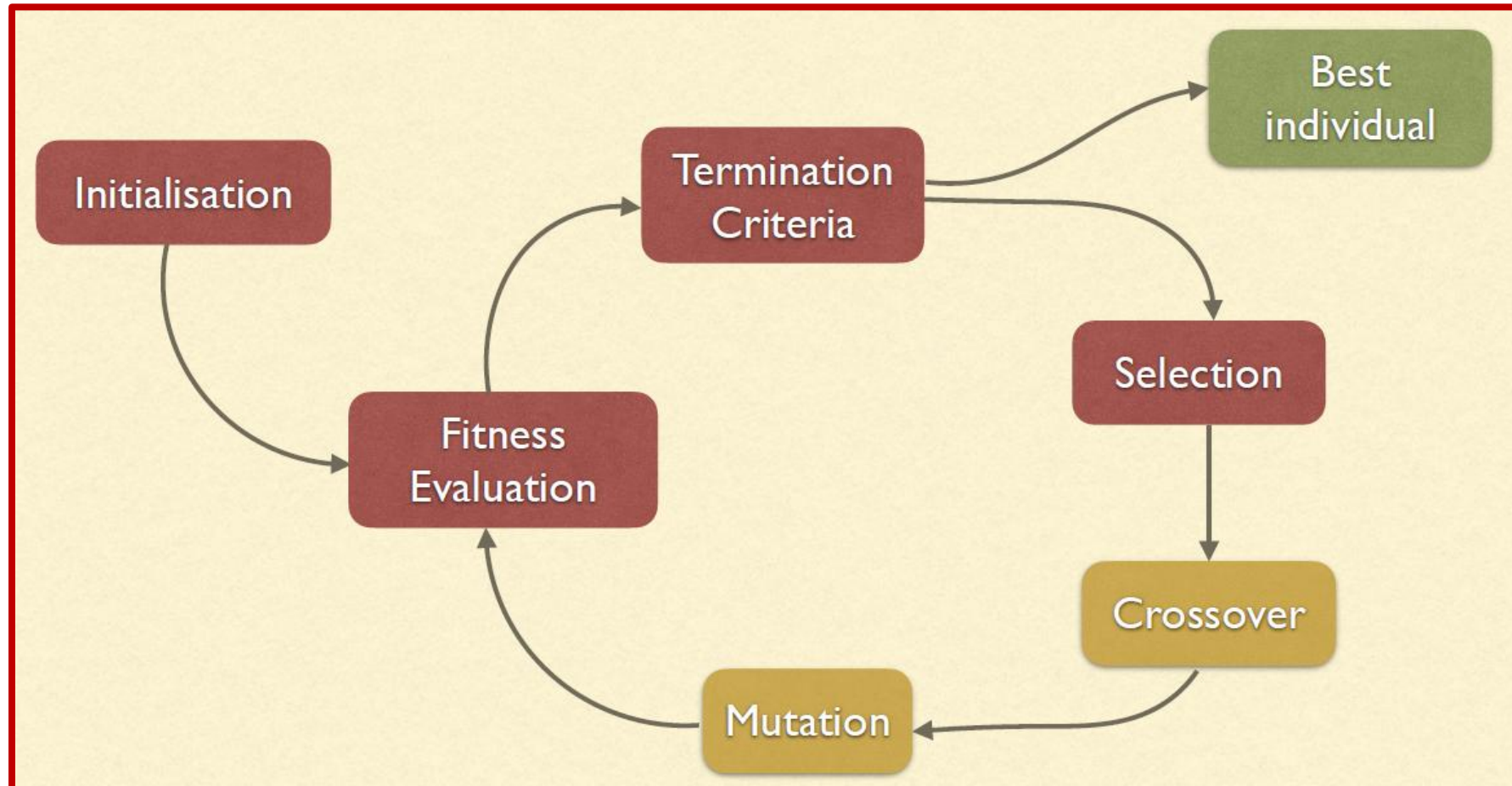


- Creator of **Genetic Programming**, i.e., the idea of... **programming computers by using evolution**

Genetic Programming

- **Genetic Programming** (GP) is a technique to stochastically evolve a population of individuals encoding **computer programs**
- John Koza was one of the firsts to talk about evolving computer **programs** in the late 80s
- Although the main “**evolutionary cycle**” is the same of GAs...
- ...GP has many peculiar properties, starting from the **representation** used

Evolutionary cycle



Programs representation

- Hold on a sec... How can we represent **programs that evolve?**

Programs representation

- Hold on a sec... How can we represent **programs that evolve?**

Parent I

```
if (A>0):  
    do_foo()
```

Parent II

```
while (x!=4):  
    do_paz()
```


Programs representation

- Hold on a sec... How can we represent **programs that evolve**?

Parent I

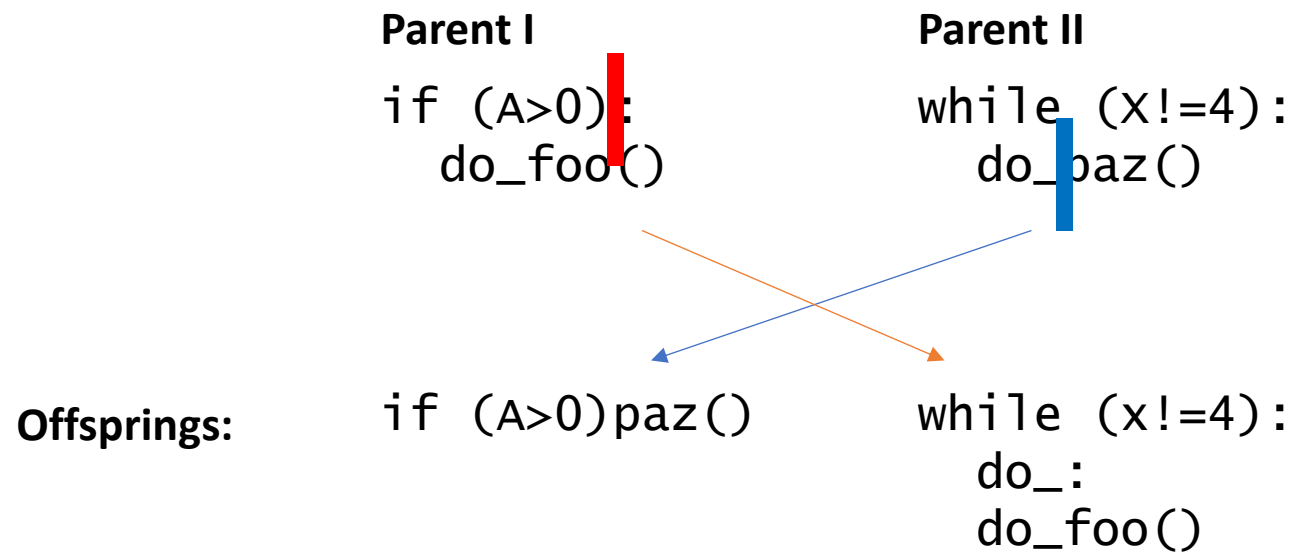
```
if (A>0):  
    do_foo()
```

Parent II

```
while (x!=4):  
    do_baz()
```

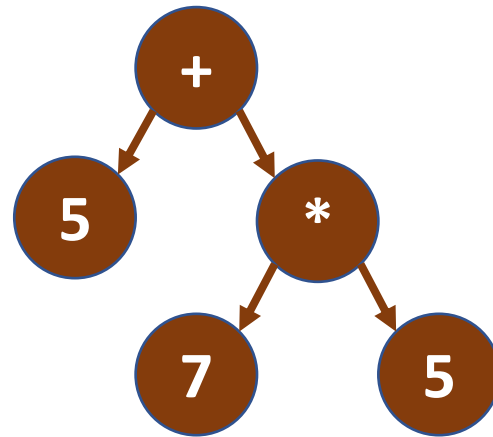
Programs representation

- Hold on a sec... How can we represent **programs that evolve**?

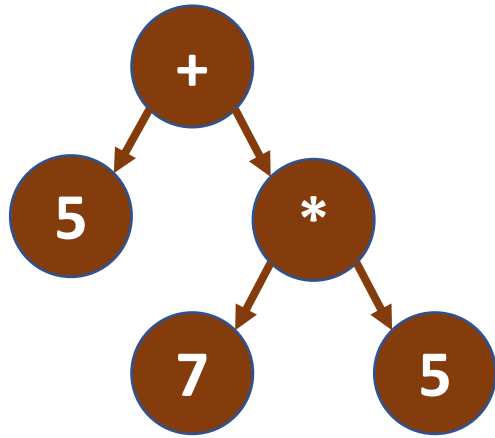


Programs representation: take 2

- We have to choose a representation where **crossover and mutation can work**
 - Which means that “a string” is usually not the best choice
- If we take an expression like “5 + (7 * 2)” we can represent it as its **parsing tree**:



Parsing trees



- Crossover and mutation can be written as **operations on subtrees**
- **Evaluation** of the tree is straightforward
- What do we need to **encode a program** as a tree?
- What are the possible **internal nodes** and **leaves**?

Terminal sets

- It contains all possible **leaves** of the tree, e.g.:

- **Constants**

- $\{0, 1, 2, \dots\}$
- $\{\text{True}, \text{False}\}$
- $\{e, \pi, -1, i\}$

- **Input variables**

- $\{x_0, x_1, \dots\}$



This is the first
difference wrt GAs!

Function set

- The function set contains all possible internal nodes, e.g.:

- **Arithmetical operations** $\{+, -, \times, \div\}$

- **Trigonometric functions** $\{\cos, \sin, \tan\}$

- **Boolean operators** $\{\wedge, \vee, \neg\}$

- **Choice conditionals** $\{\text{if... then... else}\}$

This is also a huge difference with respect to GAs!

Closure

- The primitives sets (both functionals and terminals) must respect the **property of closure**
 - Intuitively, we must be able to “mix” the primitives without problems
- **Type consistency**: the terminals and the output of any functional **must be valid inputs to all functionals**
- **Evaluation safety**: primitives that can fail at runtime (e.g., division) should be “protected” to avoid runtime failures

Sufficiency

- To find a solution **we must be able to represent it!**
 - That implies that the primitives must be sufficient to write a solution
 - Example: with real constants, variables, and $\{+, -, *, \}$ we can represent any polynomial, which is not very useful if the function that we must fit is an exponential
- Usually, **it is difficult to assure sufficiency**, but we might still obtain solutions that are good approximations
- By adding functional nodes, **we increase the sufficiency** but also the **size of the search space** → that makes the problem **more and more difficult**

Initialization

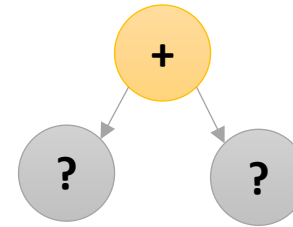
- For binary strings (i.e., traditional GA) a random generation is easy: select each bit independently with a uniform probability
- However, finite trees are infinite in number...
- Also, *“random numbers should not be generated with a method chosen at random”* (cit. Donald Knuth), which also applies to randomly generated trees
- Let’s see three classic methods for generating (valid) random trees: GROW, FULL, and RAMPED HALF & HALF

Grow

- Up to a maximum depth, select randomly across all primitives. Once the maximum depth is reached select a terminal.
- Assume $F = (+, *, -, \div)$, $T = \{x, y, 1\}$
- Max tree depth = 2

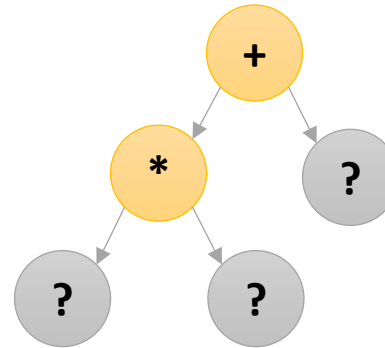
Grow

- Up to a maximum depth, select randomly across all primitives. Once the maximum depth is reached select a terminal.
- Assume $F = (+, *, -, \div)$, $T = \{x, y, 1\}$
- Max tree depth = 2



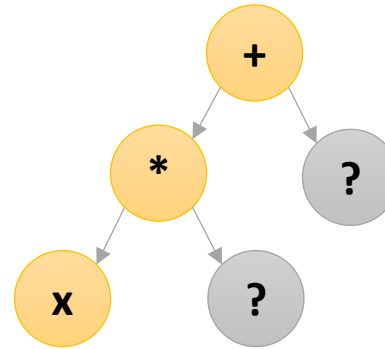
Grow

- Up to a maximum depth, select randomly across all primitives. Once the maximum depth is reached select a terminal.
- Assume $F = (+, *, -, \div)$, $T = \{x, y, 1\}$
- Max tree depth = 2



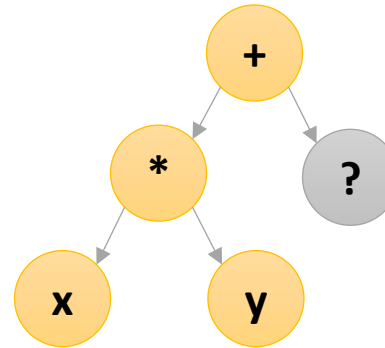
Grow

- Up to a maximum depth, select randomly across all primitives. Once the maximum depth is reached select a terminal.
- Assume $F = (+, *, -, \div)$, $T = \{\mathbf{x}, y, 1\}$
- Max tree depth = 2



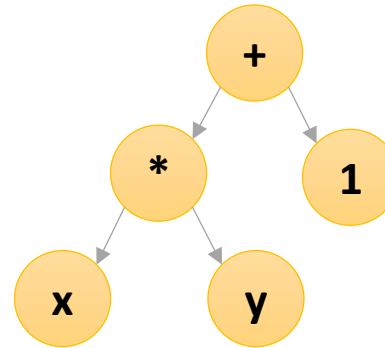
Grow

- Up to a maximum depth, select randomly across all primitives. Once the maximum depth is reached select a terminal.
- Assume $F = (+, *, -, \div)$, $T = \{x, y, 1\}$
- Max tree depth = 2



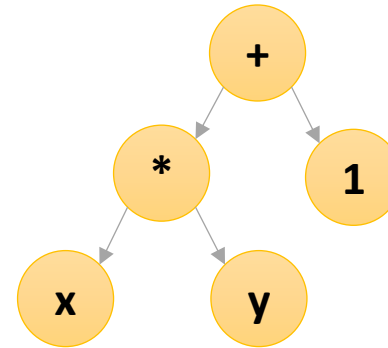
Grow

- Up to a maximum depth, select randomly across all primitives. Once the maximum depth is reached select a terminal.
- Assume $F = (+, *, -, \div)$, $T = \{x, y, 1\}$
- Max tree depth = 2



Grow

- Up to a maximum depth, select randomly across all primitives. Once the maximum depth is reached select a terminal.
- Assume $F = (+, *, -, \div)$, $T = \{x, y, \mathbf{1}\}$
- Max tree depth = 2



Tree corresponds to:
 $xy + 1$

Full

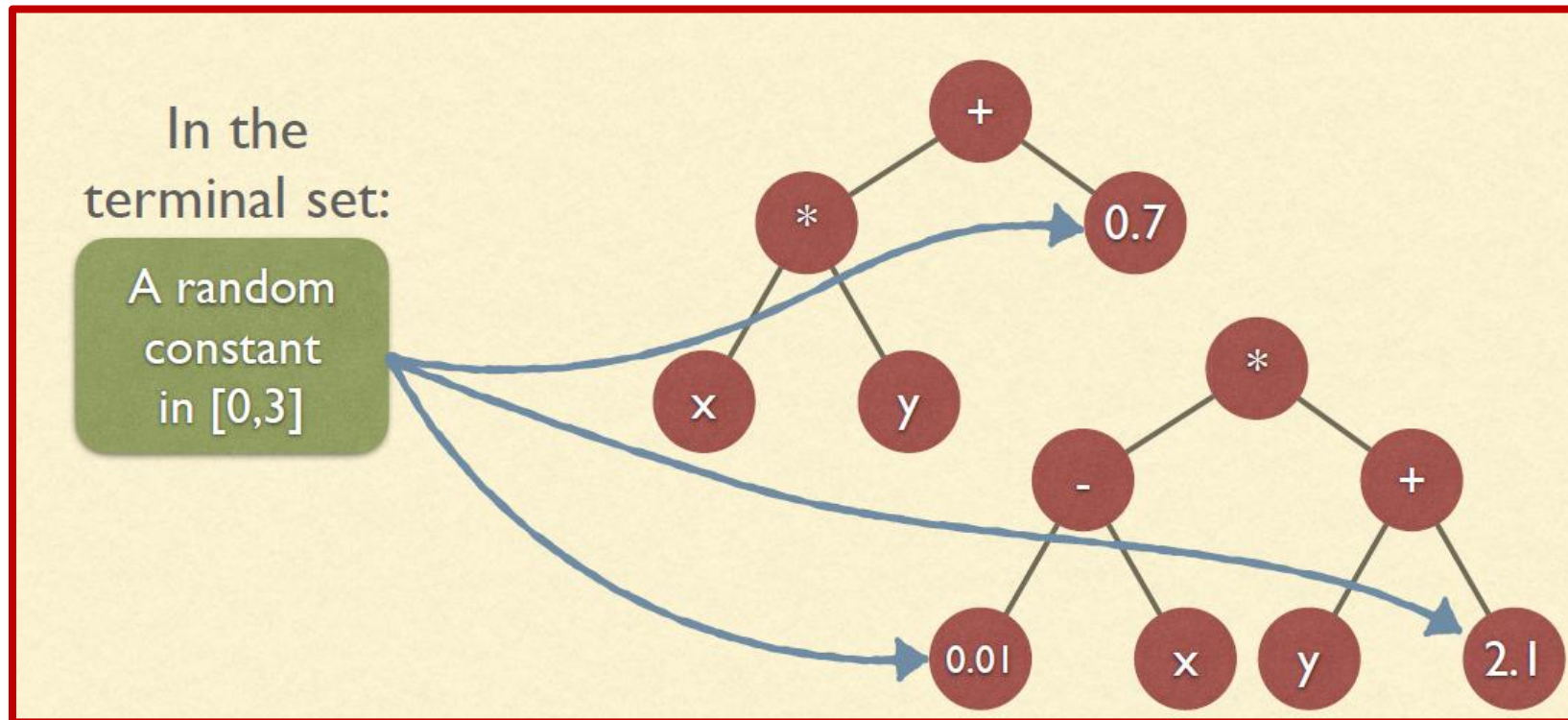
- Similar to the “grow” method, but **only functional symbols are selected** before reaching the maximum depth
 - This means that **terminals** only appear in the **last level** of the tree
- Full creates a **very different distribution of trees** w.r.t. the “grow” method, which are in general **much smaller**

Ramped half'n'half

- **Randomly select between “grow” and “full”**, with a random maximum depth chosen between a minimum and a maximum value
- It generates a population of trees characterized by a **better diversity** among them (i.e., they do not represent similar functions)

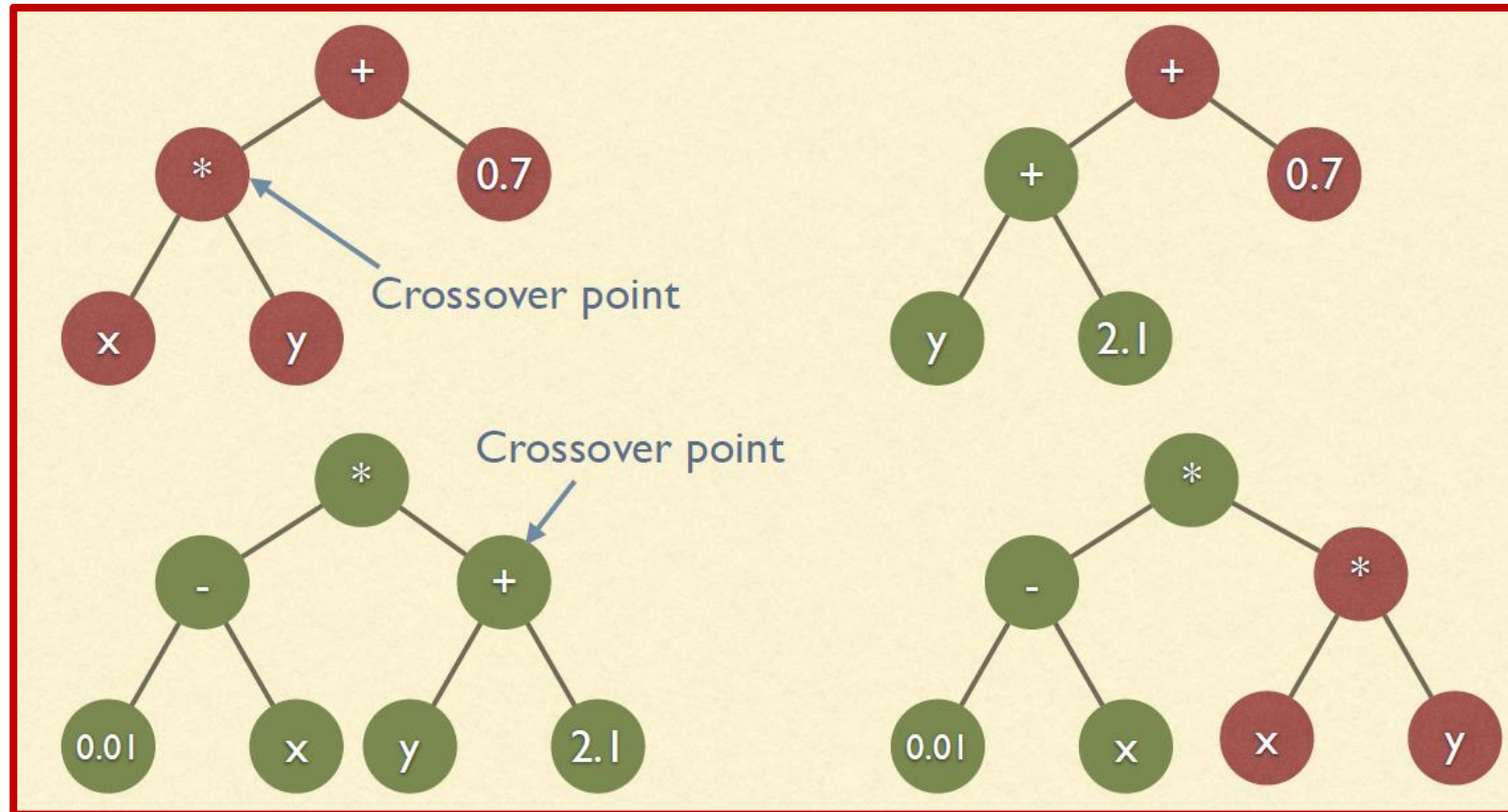
Ephemeral constants

- The terminals might include constant. But how do we choose them?
- An alternative to the choice is the use of **ephemeral constants**



Crossover

- Crossover **swaps two random subtrees** between two selected individuals

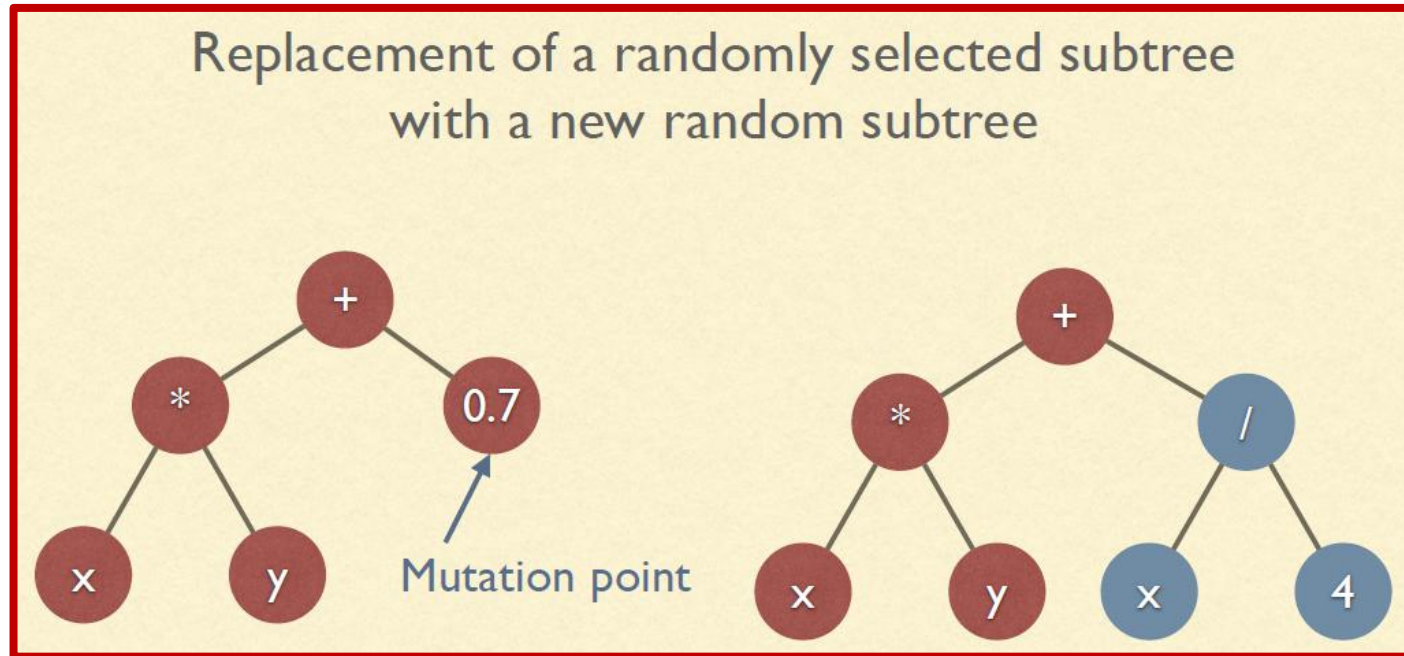


Subtleties...

- In general, **a maximum depth to the derivation tree is forced** to prevent the «bloating phenomenon» (more about this later)
- However, **crossover can produce an offspring exceeding such limit**
- There are some **special versions of GP** that prevent that by design (e.g., Grammatical Evolution) but we do not cover them here

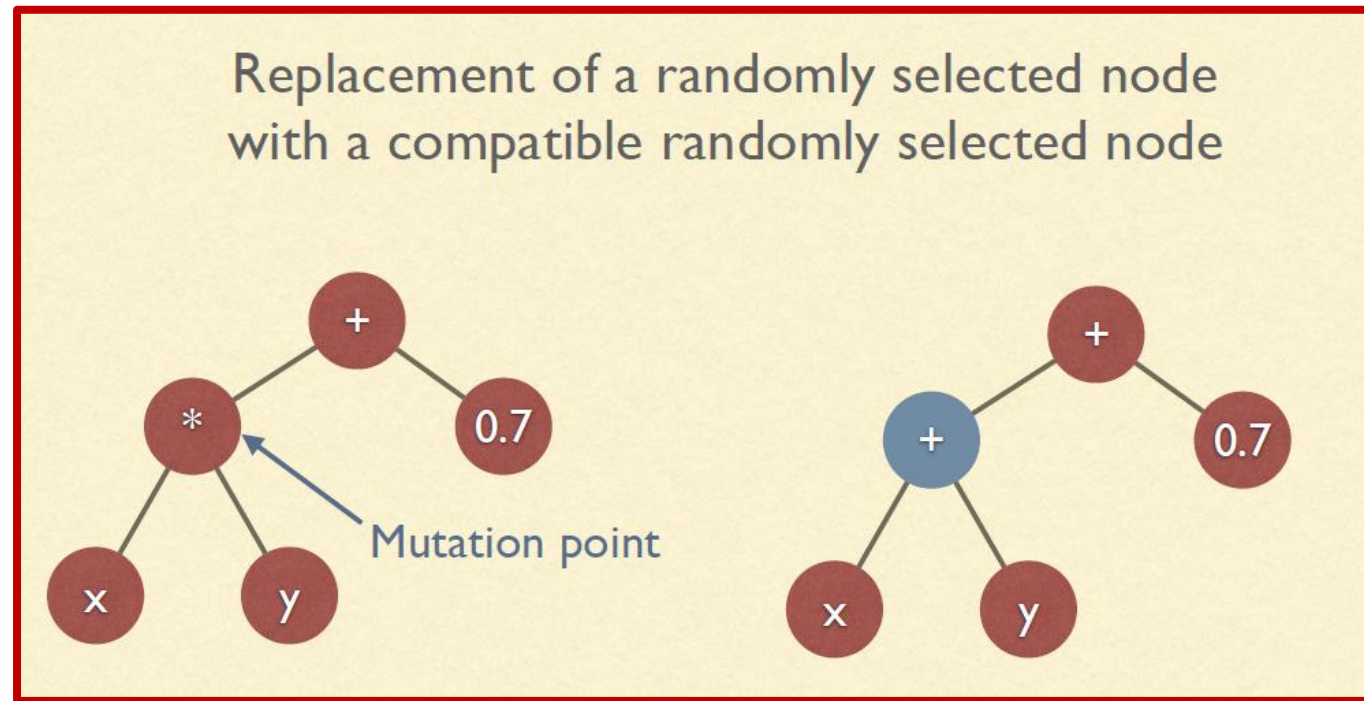
Subtree mutation

- Replaces a randomly selected subtree with a **new random subtree**



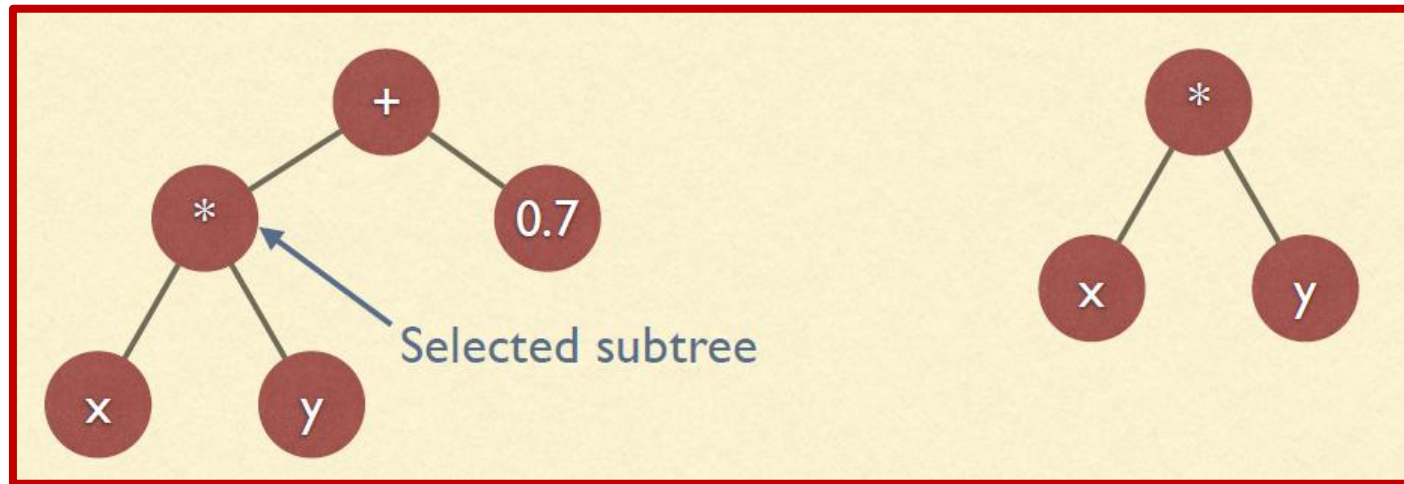
Point mutation

- Replaces a randomly selected node with a **compatible randomly selected node**

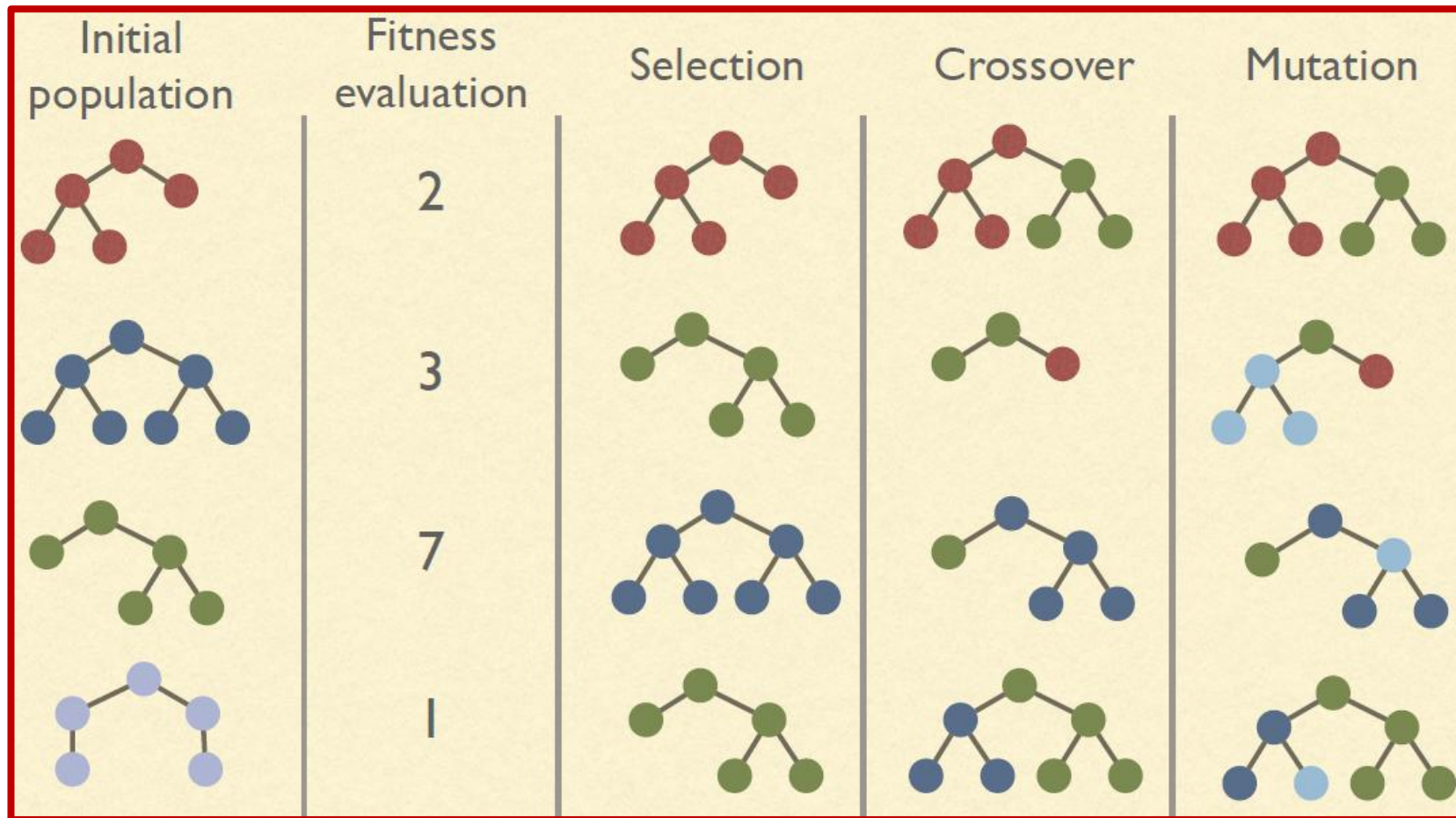


Hoist mutation

- Replaces the entire tree with one of its subtrees: **useful to reduce program size**



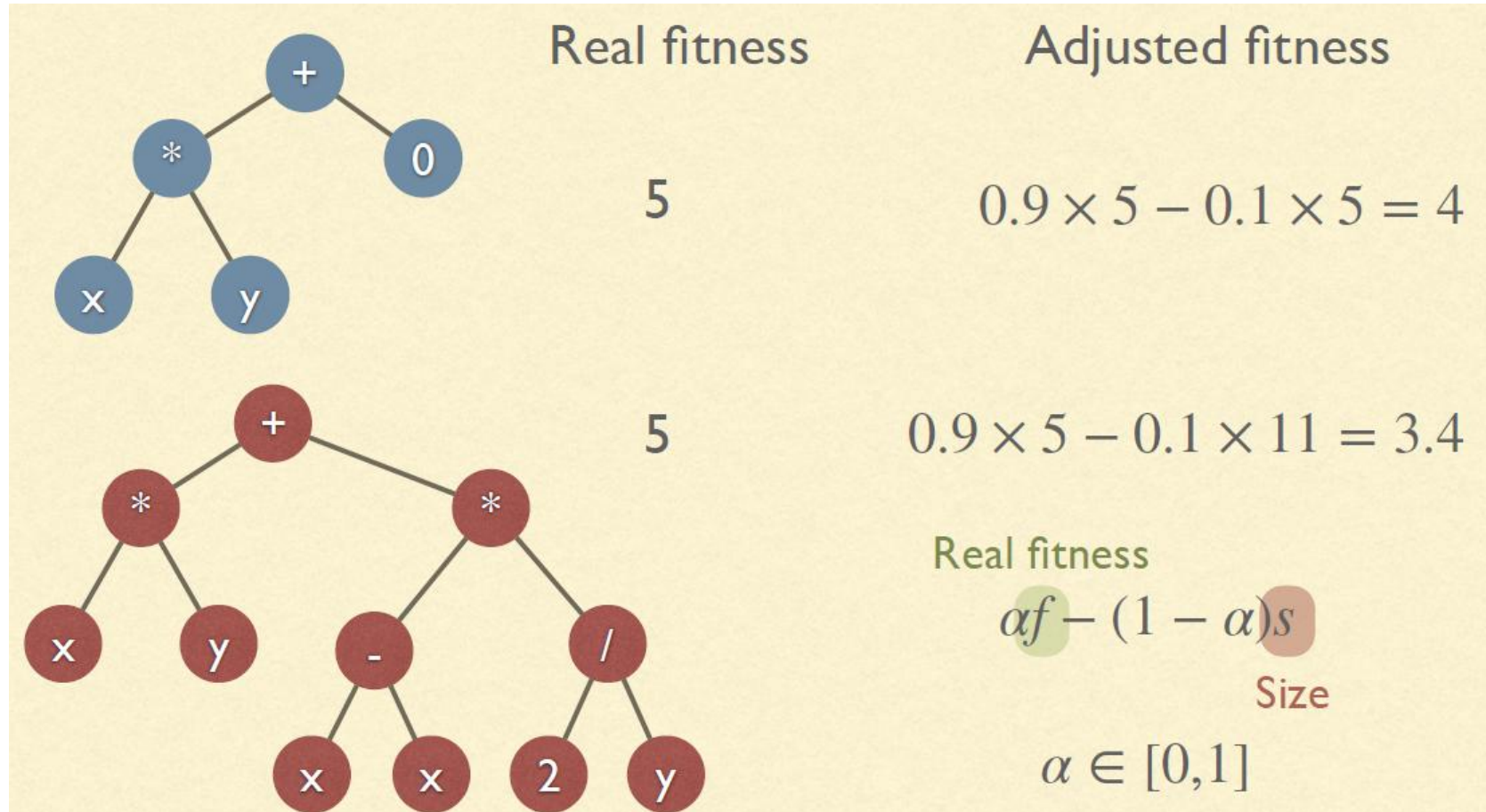
Example of GP generation



What is bloat?

- The definition of bloat is not simple, let's focus on what can be observed:
 - Large non-coding regions: many operations on the tree do not change the function that it represents
 - **Increment in the size** of the trees without a noticeable improvement in the fitness
- As a consequence, fitness evaluation is slower, **slowing down the entire GP process**
- What to do? Some heuristics:
 - **Limit the depth** of trees
 - **Implement special operators** that limit the size of offsprings
 - Identify and **remove non-coding regions** (e.g., multiplications by zero)
 - **Give a penalty** to the fitness of «big» programs

«Parsimony pressure»

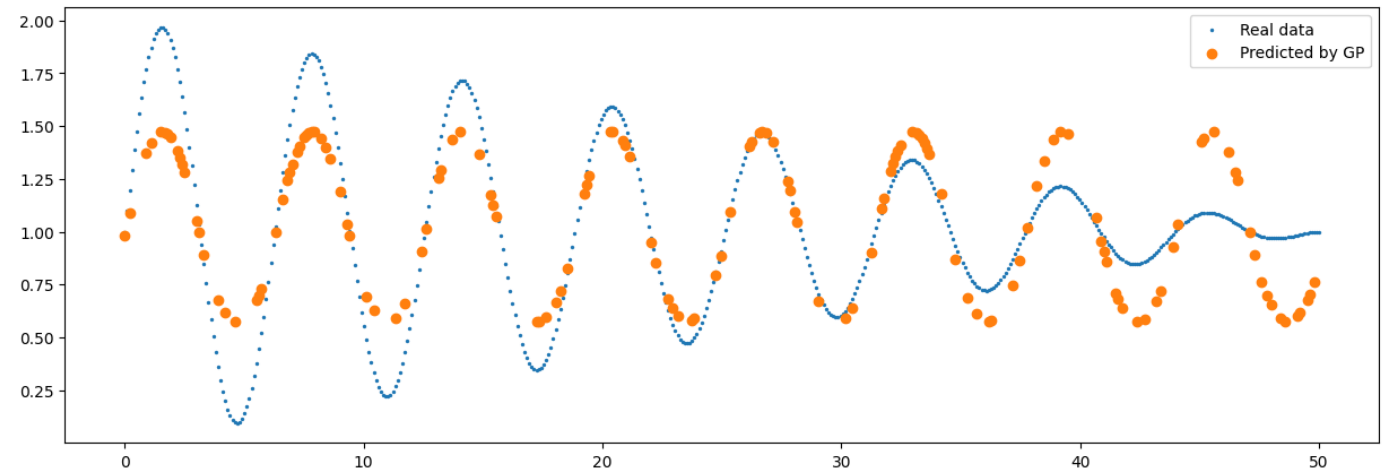
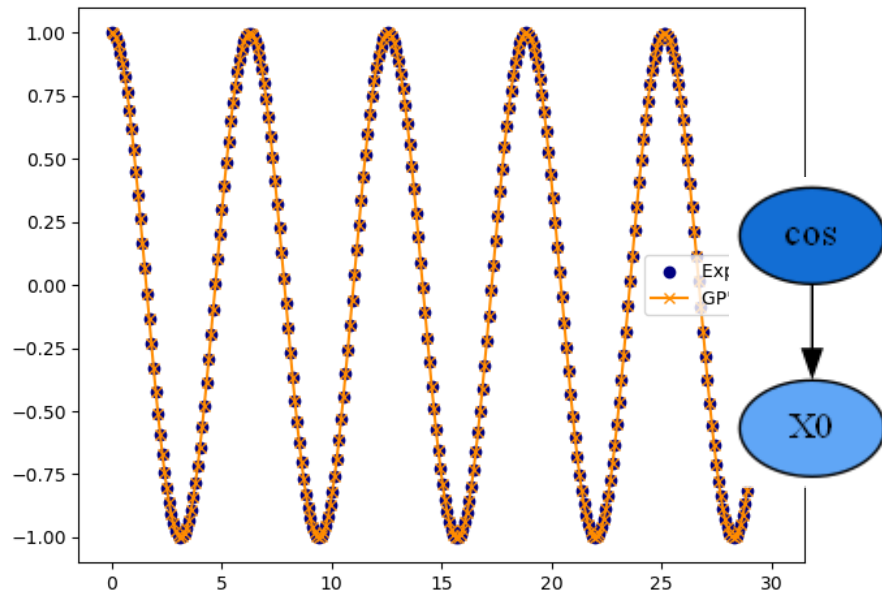


Killer application of GP

- GP does not solve an instance of a problem but it is **able to solve a «class» of problems** thanks to the use of **variables** in programs
- The killer application of GP is **symbolic regression**
 - In common regression, **we are looking for the parameters** of a model that fit with data. The model is given (e.g., the ODEs are known)
 - In symbolic regression, **we are evolving the equations** (and they do not need to be linear! That's why we have the issue of sufficiency)
- We can see this application as a special form of **machine learning**: we present **experimental data** and we obtain an **(interpretable) model**

The Return of the Harmonic Oscillator

- Let's check the (not so simple...) example with the **gplearn** library
- <https://colab.research.google.com/drive/1VBNp6tOR-X3rCLxohMj1CaEEucFghoh4?usp=sharing>

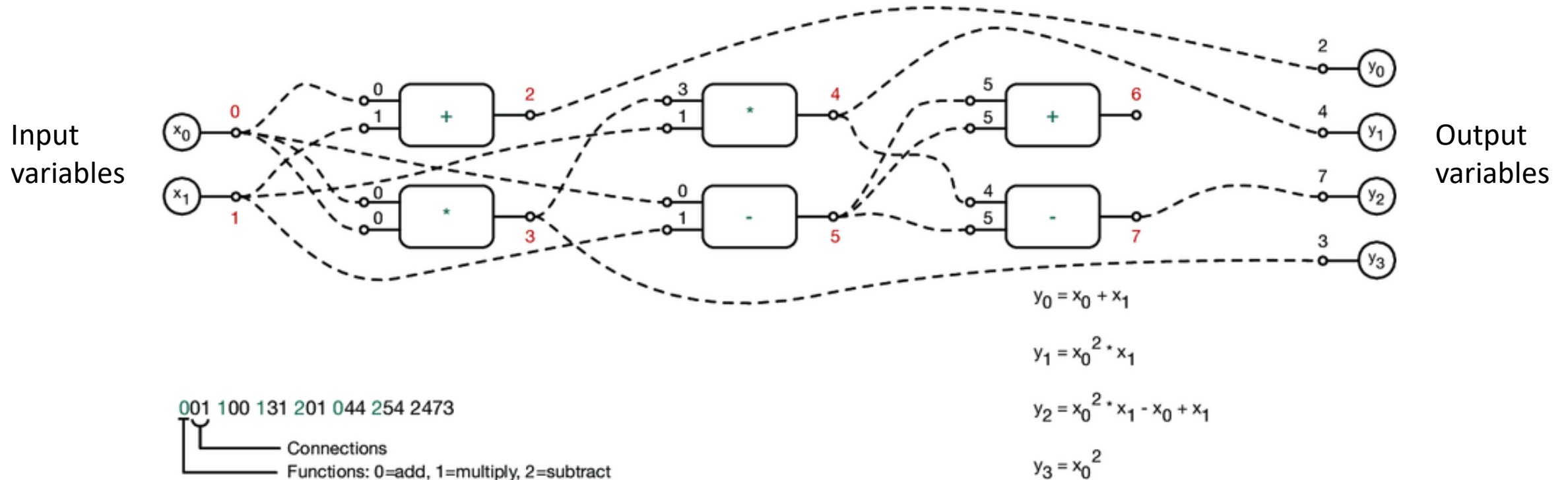


Applications of GP

- Besides symbolic regression, you can apply GP every time you need to evolve structures with a complex representation
 - **Digital circuits** (in particular, using the Cartesian GP variant)
 - Even **Quantum circuits** (see, e.g., [\[Rubinstein 2002\]](#))
 - I used GP to evolve biochemical networks (see, e.g., [\[Nobile et al., CEC 2013\]](#))
 - GP can be used to evolve **molecules and drugs** (e.g., in the form of SMILES or SELFIES)
- You can create **surrogate physical models** or **surrogate fitness landscapes**
- You can **evolve neural networks** (it is called **neuroevolution**, see) so you do not need to design the architecture by hand (see [\[Stanley et al., Nature 2019\]](#))
- You can even evolve **pipelines of ML methods** (including neural networks) using GP (it is a variant of **AutoML**)

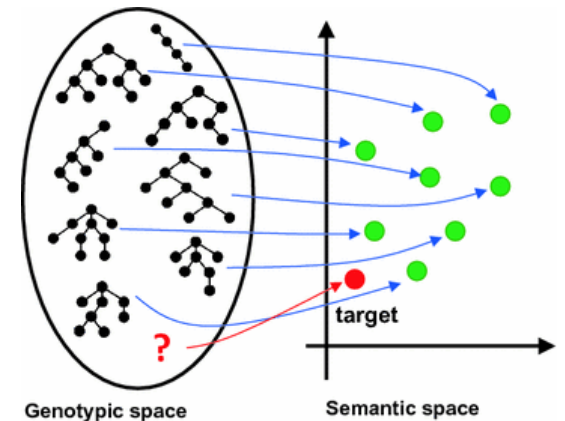
Cartesian GP

- Designed to **evolve circuits** or similar structures
 - Linear genotype**: numbers encode **connections** on a regular grid
 - <https://www.youtube.com/watch?v=qb2R0rL4OHQ> [Miller *et al.*, LNCS 2000]



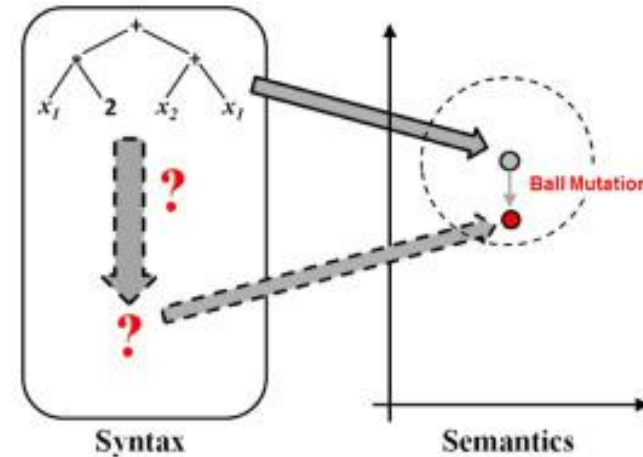
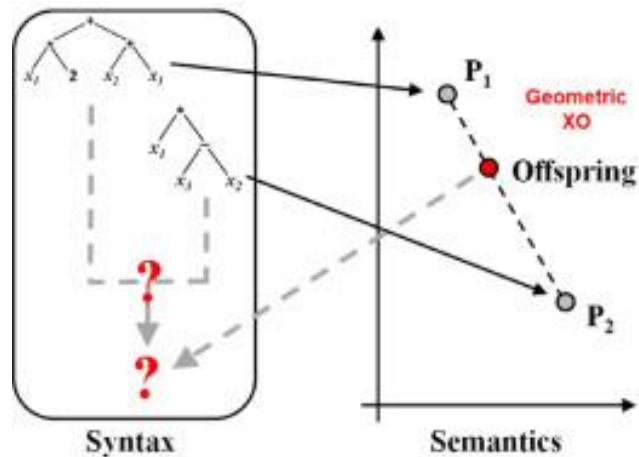
Geometric Semantic GP

- Let's assume a regression problem and let's consider a dataset with n samples $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$ with the target values $\mathbf{y} = [y_1, y_2, \dots, y_n]$
- We can run a GP program P over the samples, obtaining a vector of values $[P(\mathbf{x}_1), P(\mathbf{x}_2), \dots, P(\mathbf{x}_n)]$ which is called the ***semantic vector*** $\mathbf{s}_P$
- Both $\mathbf{y}$ and $\mathbf{s}_P$ can be seen as points in a n -dimensional «**semantic**» space
- What is we define **new genetic operators working in the semantic space?**



Semantic Genetic Operators

- Genetic semantic crossover (GSC) **generates only one offspring**, whose semantic (i.e., resulting point in semantic space) is a **linear interpolation of the parents**
- Genetic semantic mutation (GSM) produces a **new individual** whose semantic is contained in a **ball** of some given radius, centered in the semantics of the parent



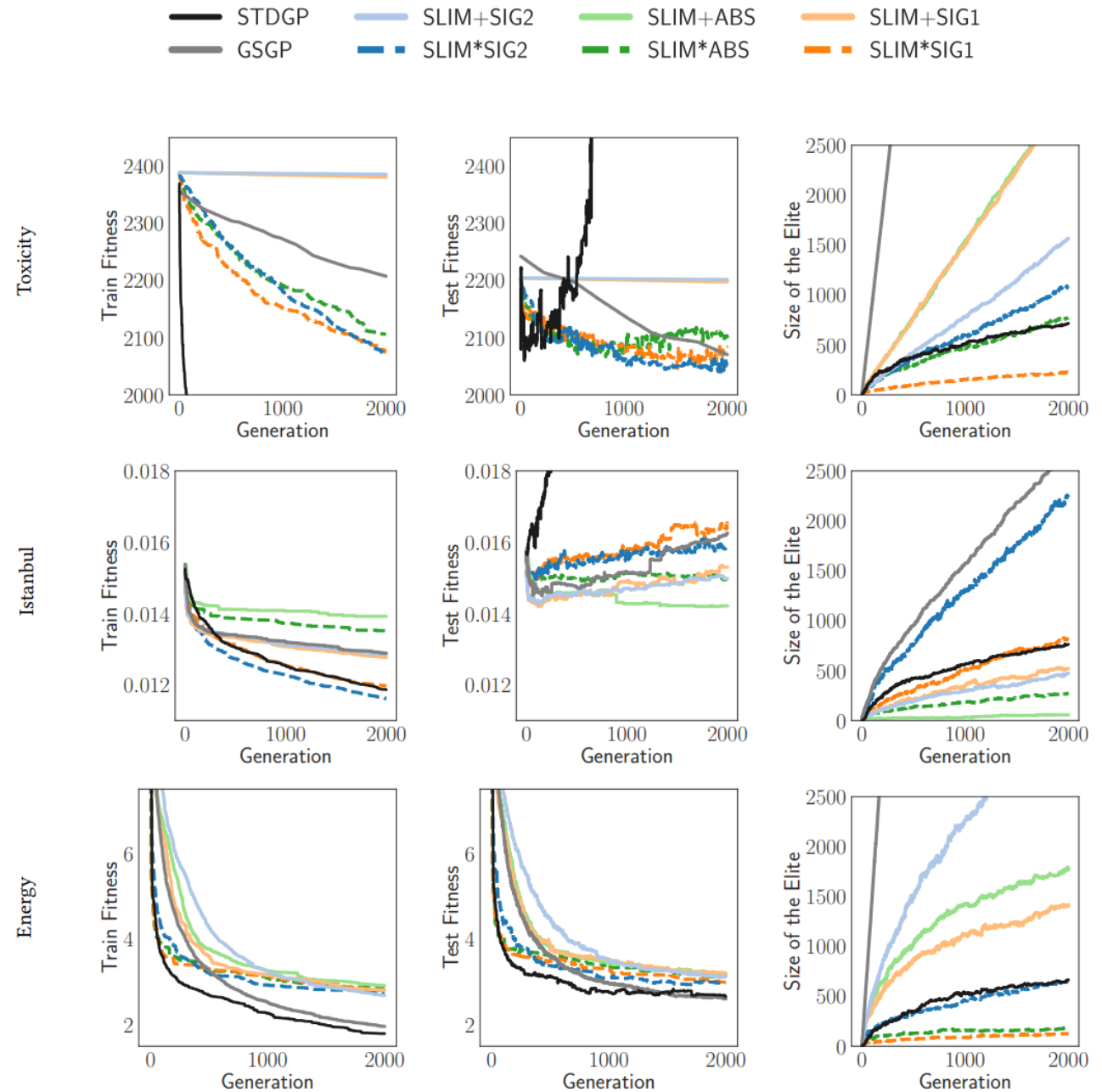
Bloating of GSGP programs

- GSC takes two parents and creates a third individual which is the **linear combination** of the two
 - That means that, iteration after iteration, **the individuals always increase their size** leading to **severe bloating**
 - Several solutions were proposed: pruning, local search, elitism, subtree approximation [Nguyen and Chu, Swarm Evol Comput 2020], ecc.
- A new version of GSGP was recently introduced [Vanneschi et al., 2024], called SLIM_GSGP: **Semantic Learning with Inflate and deflate Mutation**
 - <https://github.com/DALabNOVA/slim>

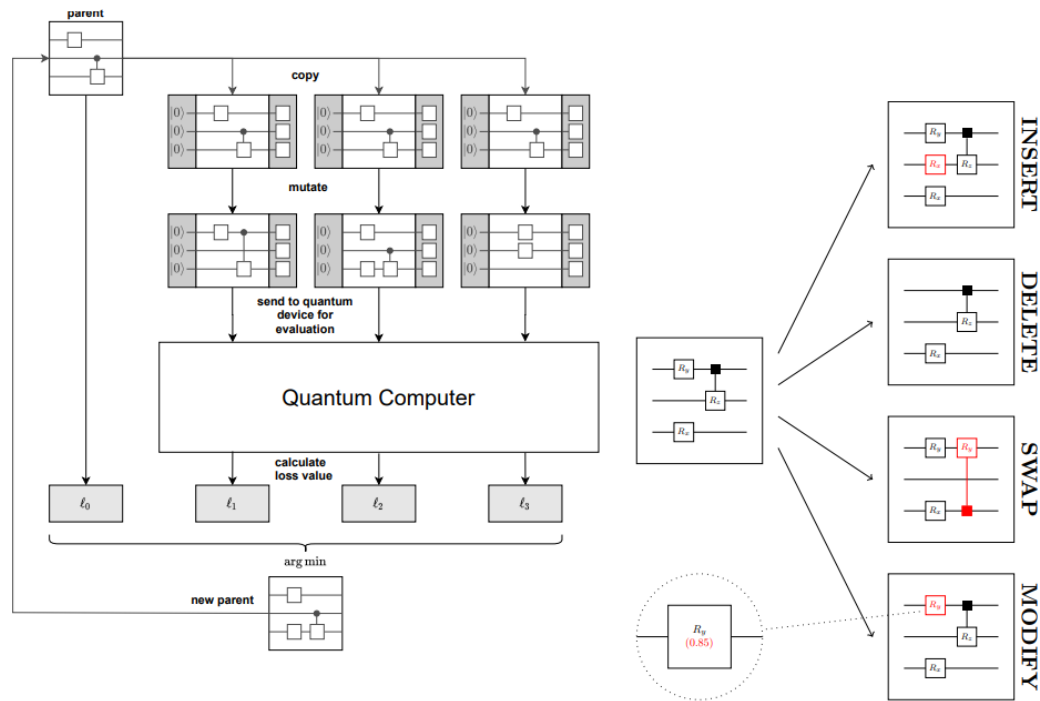
SLIM_GSGP in action

- SLIM_GSGP's mutation reuses random trees
- SLIM_GSGP's mutation «removes» repeated elements of the solution which would cancel out
- These two simple strategies are effective against bloat →

[Rosenfeld *et al.*, GECCO '25]



Evolving quantum circuits with GP



Variational quantum circuits

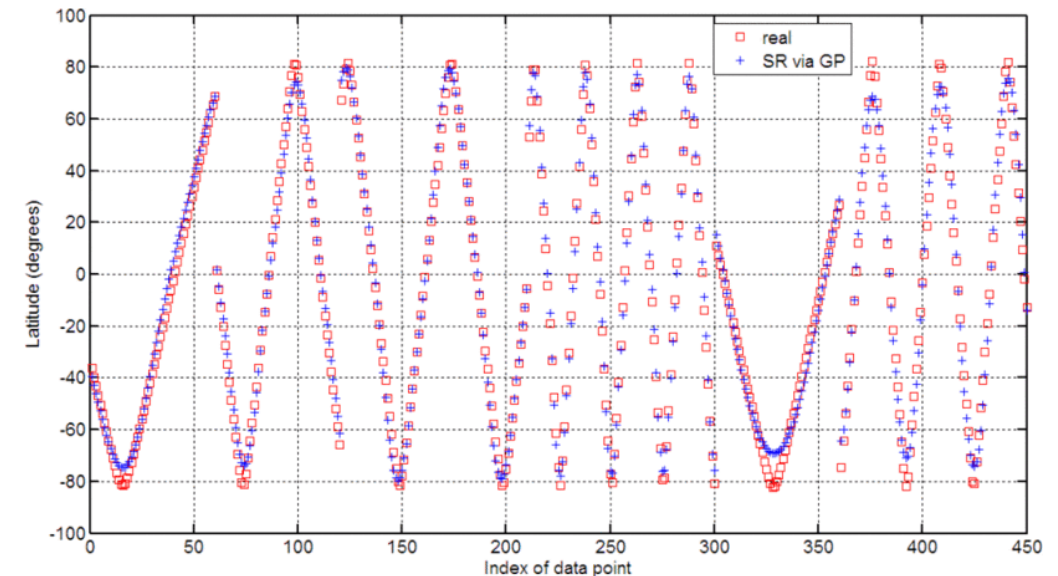
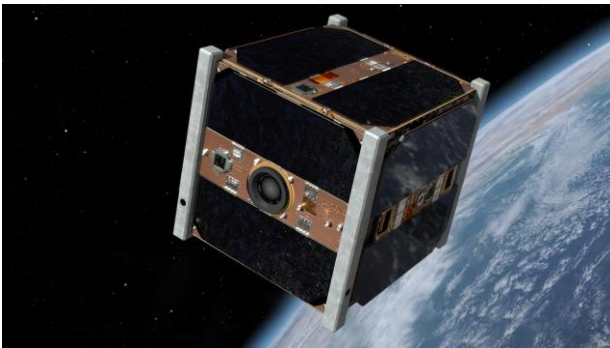
- To simplify the problem of designing quantum circuits, the **weights** of a parametrized quantum circuit are varied until the empirical sampling distribution of the circuit is sufficiently close to a desired outcome
- Alternatively, we can **optimize both the circuit design and the weights**

Learning surrogates with GP

- Instead of evaluating a real model (e.g., ODEs) **we create a surrogate with GP**
 - We take a **few samples of the fitness**, build the surrogate model and **study the surrogate**
 - The surrogate is much **faster to evaluate**
 - We can also **test «what if» scenarios**

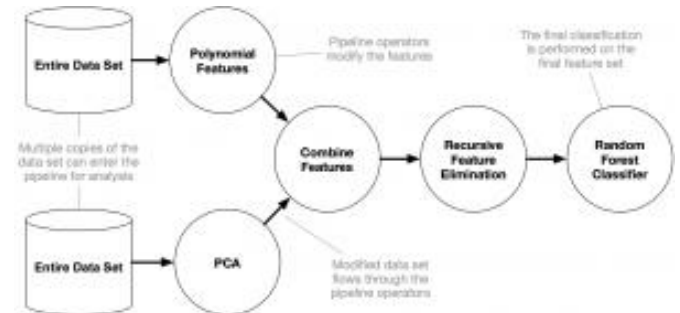
Orbital problem

The problem consists in approximating the longitude and latitude a Low Earth Orbit (LEO) satellite. The original data (input data for building the surrogate model) is some of the data available in the Two Line Element (TLE) files (keplerian elements) plus the time after the TLE creation (“epoch”), as shown in [table 1](#). The total amount of data points is 450 points. That covers 2 different pass for each one of these 3 satellites: SwissCube, BEESAT and Oscar 66 (CO-66). The reason this problem was chosen is discussed later, in subsection “Significance of these problems”



AutoML with TPOT

- **Tree-based Pipeline Optimization Tool (TPOT)**
- Automated generation of **ML pipelines**
- Uses a **tree representation** for pre-processing, data cleaning, feature manipulation, model building, etc.
- **Evolves the tree with GP**
- Available as **python library**
- See, e.g., [\[Olson *et al.*, GECCO 2016\]](#)



Summary

- Genetic Programming is an evolutionary methodology for evolving computer programs
- Basic formulation uses a derivation tree representation
- Killer-application: symbolic regression
- Can be applied to any problem that involve variables, to tackle classes of problems, or problems whose representation is too complex to use a GA
- Finally, thanks to Prof. Luca Manzoni (@UNIT5) who kindly supported the creation of this lecture